CYBERSECURITY INTERNSHIP PROGRAM

Social Engineering Attack Simulations Using Python — 15-Day Task Calendar

Issuing Body	Sqrock IT Solution
Program	Alpha 2 Internship — Cybersecurity Track
Duration	15 Days (3 Weeks)
Mode	Hands-on Python Labs + Written Reports
Prerequisite	Basic Python • Basic Networking
Legal Notice	All tasks ONLY in authorized lab environments

DIFFICULTY LEVELS			
■ Beginner	■ Intermediate	■ Advanced	■ Expert

■ 15-DAY TASK OVERVIEW

Day	Topic	Category	Difficulty
1	OSINT & Passive Reconnaissance	Week 1	Beginner
2	Email Harvesting & Social Engineering Prep	Week 1	Beginner
3	Phishing Page Anatomy & Detection	Week 1	Beginner
4	Vishing & Smishing Simulation Scripts	Week 1	Beginner
5	OSINT + SE: Build a Target Profile	Week 1	Intermediate
6	Spear Phishing Email Craft (Lab Only)	Week 2	Intermediate
7	Password Attacks & Credential Stuffing	Week 2	Intermediate
8	USB Drop Attack Simulation	Week 2	Intermediate
9	Social Media Impersonation & Fake Profile Detection	Week 2	Intermediate
10	Baiting & Watering Hole Attack Simulation	Week 2	Intermediate
11	Social Engineering Awareness Training Module	Week 3	Intermediate
12	Phishing Email Detection with ML	Week 3	Advanced
13	SIEM Log Analysis for SE Attack Detection	Week 3	Advanced
14	SE Incident Response Plan	Week 3	Advanced
15	Final Project: SE Attack Chain Simulator	Week 3	Expert

WEEK 1 — RECON & OSINT

DAY 1

OSINT & Passive Reconnaissance

■ Beginner

■ OBJECTIVE

Understand passive info-gathering. No direct target contact.

■ THEORY

- OSINT = Open-Source Intelligence. Uses public data only.
- Key sources: WHOIS, DNS records, Shodan, LinkedIn, GitHub leaks.
- Difference between passive (no contact) vs active (direct probe) recon.

■ TASK

Build a Python OSINT scanner that collects WHOIS, DNS, and IP geolocation for a domain.

</> PYTHON

```
import whois, socket, requests

def osint_scan(domain):
    w = whois.whois(domain)
    ip = socket.gethostbyname(domain)
    geo = requests.get(f"http://ip-api.com/json/{ip}").json()
    print(f"Registrar : {w.registrar}")
    print(f"IP        : {ip}")
    print(f"Location  : {geo['city']}, {geo['country']}")

osint_scan("example.com")
```

■ Tools: python-whois | requests | socket

■ Deliverable: Script output screenshot + 1-page OSINT report on a practice domain.

DAY 2

Email Harvesting & Social Engineering Prep

■ Beginner

■ OBJECTIVE

Harvest emails ethically and understand pretexting fundamentals.

■ THEORY

- Email harvesting: collecting addresses from public web sources.
- Pretexting: crafting a believable fake identity for SE attacks.
- Ethics boundary: only target domains you own or have written permission for.

■ TASK

Write a Python script to scrape emails from a website (your own/lab domain) using regex.

</> PYTHON

```
import requests, re

def harvest_emails(url):
    html = requests.get(url, timeout=10).text
    emails = set(re.findall(r'[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}', html))
    return emails

found = harvest_emails("http://testphp.vulnweb.com")
for e in found:
    print(e)
```

■ Tools: requests | re (stdlib)

■ Deliverable: Script + list of found emails + short write-up on how attackers use this.

DAY 3

Phishing Page Anatomy & Detection

■ Beginner

■ OBJECTIVE	Analyze real phishing pages. Build a detector, NOT a phishing page.
■ THEORY	• Phishing relies on visual cloning + urgency triggers. • URL tricks: homograph attacks, subdomain abuse (paypal.login.evil.com). • Indicators of phishing: mismatched domain, HTTP (no TLS), form action to external URL.
■ TASK	**Write a URL phishing scorer in Python that flags suspicious domains.**

</> PYTHON

```
import re
from urllib.parse import urlparse

KEYWORDS = ["login","verify","secure","update","account","bank","paypal"]

def phish_score(url):
    p = urlparse(url)
    score = 0
    if not url.startswith("https"): score += 30
    for kw in KEYWORDS:
        if kw in p.netloc: score += 20
    if p.netloc.count('.') > 3:     score += 25
    if re.search(r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', p.netloc):
        score += 40
    return min(score, 100)

urls = ["https://paypal-login.evil.com/verify","https://github.com"]
for u in urls:
    print(f"{u} -> Risk: {phish_score(u)}%")
```

■ Tools: urllib (stdlib) | re (stdlib)

■ **Deliverable: Script + test results on 10 sample URLs + explanation of each scoring factor.**

DAY 4

Vishing & Smishing Simulation Scripts

■ Beginner

■ OBJECTIVE

Model voice/SMS social engineering scripts for awareness training.

■ THEORY

- Vishing = voice phishing. Attacker impersonates bank, IT, or government.
- Smishing = SMS phishing. Short links + urgency = high click rate.
- Psychological triggers: authority, scarcity, fear, liking.

■ TASK

Build a Python-based SE script generator that produces awareness-training call scripts.

</> PYTHON

```
def generate_vishing_script(target_company, attacker_role, pretext):
    script = f"""
=== VISHING AWARENESS SCRIPT ===
Caller Role  : {attacker_role}
Target Org   : {target_company}
Pretext      : {pretext}

[OPENER]
'Hi, this is Alex from IT Support at {target_company}.
We detected unusual activity on your account.'

[HOOK]
'I need to verify your identity — can you confirm
your employee ID and current password?'

[RED FLAG for Awareness]
-> Legitimate IT will NEVER ask for passwords.
-> Always verify via official internal channels.
"""
    return script

print(generate_vishing_script("Sqrock IT", "IT Support", "Password Reset"))
```

■ Tools: Python (stdlib)

■ Deliverable: 3 unique scripts (IT, bank, gov) + analysis of psychological triggers used.

DAY 5

OSINT + SE: Build a Target Profile

■ Intermediate

■ OBJECTIVE

Combine OSINT data into a social engineering threat profile.

■ THEORY

- Attackers use LinkedIn, GitHub, Twitter to build detailed target profiles.
- Profile includes: name, role, tech stack, colleagues, habits.
- Defender use: identify what's exposed and reduce digital footprint.

■ TASK

Create a Python tool that aggregates GitHub public data into a target profile JSON.

</> PYTHON

```
import requests, json

def github_profile(username):
    base = "https://api.github.com"
    u = requests.get(f"{base}/users/{username}").json()
    repos = requests.get(f"{base}/users/{username}/repos").json()
    langs = {}
    for r in repos[:10]:
        if r.get('language'):
            langs[r['language']] = langs.get(r['language'],0)+1
    profile = {
        "name"       : u.get("name"),
        "company"    : u.get("company"),
        "location"   : u.get("location"),
        "public_repos": u.get("public_repos"),
        "top_langs"  : langs,
        "bio"        : u.get("bio"),
    }
    print(json.dumps(profile, indent=2))

github_profile("torvalds")
```

■ Tools: requests | json (stdlib)

■ Deliverable: Profile JSON + 1-page 'attacker perspective' threat analysis report.

WEEK 2 — ACTIVE ATTACKS

DAY 6

Spear Phishing Email Craft (Lab Only)

■ Intermediate

■ OBJECTIVE

Craft personalized phishing emails for security-awareness training.

■ THEORY

- Spear phishing targets a specific individual using gathered OSINT.
- Key elements: spoofed sender, personal hook, malicious link/attachment.
- Defense: DMARC, SPF, DKIM — email authentication standards.

■ TASK

Build a Python email template engine that generates personalized phishing-awareness training emails.

</> PYTHON

```
def spear_phish_template(target):
    return f"""
From    : it-support@{target['company'].lower()}.com
To      : {target['email']}
Subject : Action Required: Your {target['company']} account will be disabled

Hi {target['name']},

Our security team noticed a login from {target['location']}.
Please verify your account within 24 hours to avoid suspension.

[Verify Account] -> https://lab.internal/awareness-test

Regards,
IT Security Team
"""

target = {
    "name": "Riya Sharma", "email": "riya@company.com",
    "company": "Sqrock", "location": "Bangalore, India"
}
print(spear_phish_template(target))
```

■ Tools: Python (stdlib)

■ Deliverable: 3 personalized email drafts + DMARC/SPF setup guide for defenders.

DAY 7

Password Attacks & Credential Stuffing

■ Intermediate

■ OBJECTIVE

Understand brute-force logic and build a rate-limit detector.

■ THEORY

- Credential stuffing = using leaked username:password combos on other sites.
- Brute force = systematic guess. Dictionary attack = wordlist-based.
- Defense: account lockout, CAPTCHA, MFA, breach monitoring.

■ TASK

Write a Python login bruteforce simulator against a local Flask test server.

</> PYTHON

```
# Run against YOUR OWN local lab server only
import requests

def brute_force_sim(url, username, wordlist):
    for pwd in wordlist:
        r = requests.post(url,
            data={"username": username, "password": pwd},
            timeout=5)
        if "Welcome" in r.text or r.status_code == 200:
            print(f"[+] FOUND: {username}:{pwd}")
            return pwd
        else:
            print(f"[-] Failed: {pwd}")
    return None

wordlist = ["123456","password","admin","letmein","qwerty"]
brute_force_sim("http://localhost:5000/login", "admin", wordlist)
```

■ Tools: requests | Flask (lab server)

■ **Deliverable: Script demo on local lab + defensive implementation (rate limiter in Flask).**

DAY 8

USB Drop Attack Simulation

■ Intermediate

■ OBJECTIVE	Simulate a USB drop payload (awareness) using Python autorun logic.
■ THEORY	• USB drops: attackers leave infected USB drives in target locations. • AutoRun abuse: malicious script runs when USB is inserted (Windows). • Defense: disable AutoRun, endpoint DLP, user awareness training.
■ TASK	**Build a benign Python 'payload' that logs system info on execution — simulates what a real payload would do.**

</> PYTHON

```
import platform, socket, datetime, os

def usb_payload_sim(output_file="recon_log.txt"):
    info = {
        "timestamp" : str(datetime.datetime.now()),
        "hostname"  : socket.gethostname(),
        "os"        : platform.system(),
        "version"   : platform.version(),
        "user"      : os.getenv("USERNAME") or os.getenv("USER"),
        "cwd"       : os.getcwd(),
    }
    with open(output_file, "w") as f:
        for k, v in info.items():
            f.write(f"{k}: {v}\n")
    print(f"[SIM] Recon data saved to {output_file}")

usb_payload_sim()
```

■ Tools: platform | socket | os | datetime (all stdlib)

■ **Deliverable: Simulated output file + write-up on USB drop prevention policy.**

DAY 9

Social Media Impersonation & Fake Profile Detection

■ Intermediate

■ OBJECTIVE

Detect fake/bot social profiles using behavioral heuristics.

■ THEORY

- Fake profiles are used to build trust before SE attacks.
- Bot signals: low follower:following ratio, generic name, new account.
- Impersonation: cloned profile photo, similar username to real person.

■ TASK

Write a Python fake-profile scorer that evaluates Twitter/X-like user data.

</> PYTHON

```
def fake_profile_score(profile):
    score = 0
    age_days = profile.get("account_age_days", 365)
    if age_days < 30:                              score += 30
    followers = profile.get("followers", 1)
    following = profile.get("following", 1)
    ratio = following / max(followers, 1)
    if ratio > 10:                                 score += 25
    if profile.get("no_profile_pic"):              score += 20
    if profile.get("posts", 100) < 5:             score += 15
    if profile.get("default_bio"):                 score += 10
    return min(score, 100)

profiles = [
    {"account_age_days":7,"followers":2,"following":900,
     "no_profile_pic":True,"posts":1,"default_bio":True},
    {"account_age_days":1200,"followers":4500,"following":320,
     "no_profile_pic":False,"posts":870,"default_bio":False},
]
for i, p in enumerate(profiles):
    print(f"Profile {i+1} -> Fake Score: {fake_profile_score(p)}%")
```

■ Tools: Python (stdlib)

■ Deliverable: Script + analysis of 5 real (anonymized) profile samples.

DAY 10

Baiting & Watering Hole Attack Simulation

■ Intermediate

■ OBJECTIVE

Understand drive-by download logic and build a honeypot tracker.

■ THEORY

- Baiting: physical (USB, CD) or digital (fake download link) traps.
- Watering hole: compromise a site your target frequently visits.
- Defense: web filtering, script blocking (NoScript), patch management.

■ TASK

Build a Python honeypot link tracker — logs who clicks a 'bait' link.

</> PYTHON

```
from http.server import HTTPServer, BaseHTTPRequestHandler
import datetime, json

LOG = []

class HoneyHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        entry = {
            "time"   : str(datetime.datetime.now()),
            "ip"     : self.client_address[0],
            "path"   : self.path,
            "agent"  : self.headers.get("User-Agent","?"),
        }
        LOG.append(entry)
        print(json.dumps(entry))
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"Thanks for visiting!")

    def log_message(self, *args): pass   # suppress default logs

print("Honeypot on http://localhost:8080")
HTTPServer(("", 8080), HoneyHandler).serve_forever()
```

■ Tools: http.server (stdlib) | json (stdlib)

■ Deliverable: Server demo + log analysis of captured 'victims' + mitigation report.

WEEK 3 — DEFENSE & REPORTING

DAY 11

Social Engineering Awareness Training Module

■ Intermediate

■ OBJECTIVE

Design and code an interactive SE awareness quiz in Python.

■ THEORY

- Training is the #1 defense against social engineering.
- Effective training: realistic scenarios, immediate feedback, repetition.
- Phishing simulation programs (GoPhish, KnowBe4) reduce click rates by 70%+.

■ TASK

Build a CLI quiz engine in Python with scenario-based SE questions and score tracking.

</> PYTHON

```
QUESTIONS = [
    {"q": "An email asks you to verify your password via a link. You should:",
     "opts": ["A) Click the link","B) Call IT directly","C) Reply with password"],
     "ans": "B", "exp": "Always verify via official channels, never click email links."},
    {"q": "You find a USB drive in the parking lot. You should:",
     "opts": ["A) Plug it in to check","B) Hand to security","C) Keep it"],
     "ans": "B", "exp": "USB drops are a classic baiting attack vector."},
]

def run_quiz():
    score = 0
    for i, q in enumerate(QUESTIONS, 1):
        print(f"\nQ{i}: {q['q']}")
        for o in q['opts']: print(f"  {o}")
        ans = input("Your answer (A/B/C): ").strip().upper()
        if ans == q['ans']:
            print("Correct!"); score += 1
        else:
            print(f"Wrong. {q['exp']}")
    print(f"\nScore: {score}/{len(QUESTIONS)}")

run_quiz()
```

■ Tools: json (stdlib) | Python (stdlib)

■ Deliverable: Quiz with 10+ questions + score report saved to JSON file.

DAY 12

Phishing Email Detection with ML

■ Advanced

■ OBJECTIVE

Train a basic Naive Bayes classifier to detect phishing emails.

■ THEORY

- NLP + ML can classify emails with 95%+ accuracy when trained properly.
- Features: keyword frequency, URL count, sender domain, urgency words.
- Naive Bayes is fast and works well on text classification tasks.

■ TASK

Build a phishing email classifier using scikit-learn and a small labeled dataset.

</> PYTHON

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

emails = [
    "Verify your account now or it will be suspended",
    "Click here to claim your prize immediately",
    "Team standup at 3pm, agenda attached",
    "Your invoice for Q2 is ready for review",
    "Urgent: update your bank details to avoid closure",
]
labels = [1, 1, 0, 0, 1]   # 1=phishing, 0=legit

pipe = Pipeline([
    ("vec", CountVectorizer()),
    ("clf", MultinomialNB()),
])
pipe.fit(emails, labels)

tests = ["Please verify your PayPal login", "Meeting notes from yesterday"]
for t in tests:
    pred = pipe.predict([t])[0]
    print(f"{'PHISHING' if pred else 'LEGIT'}: {t}")
```

■ Tools: scikit-learn | pandas

■ Deliverable: Trained model + confusion matrix + accuracy report on 50-sample dataset.

DAY 13

SIEM Log Analysis for SE Attack Detection

■ Advanced

■ OBJECTIVE

Parse and analyze security logs to detect social engineering patterns.

■ THEORY

- SIEM = Security Information and Event Management.
- SE leaves traces: unusual login times, failed attempts, new email rules.
- Log sources: Windows Event Log, mail gateway, web proxy, VPN.

■ TASK

Write a Python log parser that flags social-engineering-related anomalies.

</> PYTHON

```
import re
from collections import Counter

LOG_SAMPLE = """
2024-01-15 02:34:12 FAILED_LOGIN user=admin ip=45.33.32.156
2024-01-15 02:34:14 FAILED_LOGIN user=admin ip=45.33.32.156
2024-01-15 02:34:16 SUCCESS_LOGIN user=admin ip=45.33.32.156
2024-01-15 08:00:01 SUCCESS_LOGIN user=riya ip=192.168.1.10
2024-01-15 02:35:00 EMAIL_RULE_CREATED user=admin rule=forward_all
"""

def analyze_logs(logs):
    fails = re.findall(r'FAILED_LOGIN user=(\w+) ip=([\d.]+)', logs)
    rules = re.findall(r'EMAIL_RULE_CREATED user=(\w+)', logs)
    fail_counts = Counter(u for u, _ in fails)
    for user, count in fail_counts.items():
        if count >= 3:
            print(f"[ALERT] Brute force detected: {user} ({count} failures)")
    for user in rules:
        print(f"[ALERT] Suspicious email rule created by: {user}")

analyze_logs(LOG_SAMPLE)
```

■ Tools: re (stdlib) | collections (stdlib)

■ Deliverable: Parser demo + alert report on provided sample logs.

DAY 14

SE Incident Response Plan

■ Advanced

■ OBJECTIVE

Draft and automate a social engineering incident response workflow.

■ THEORY

- IR phases: Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned.
- SE incidents require: account lockout, forensic email analysis, user notification.
- Documentation is critical for legal, compliance, and insurance purposes.

■ TASK

Build a Python IR automation script that triggers containment actions on SE alert.

</> PYTHON

```
import datetime, json

def ir_response(incident):
    print(f"\n=== INCIDENT RESPONSE TRIGGERED ===")
    print(f"Time     : {datetime.datetime.now()}")
    print(f"Type     : {incident['type']}")
    print(f"Severity : {incident['severity']}")

    actions = []
    if incident['severity'] in ('HIGH','CRITICAL'):
        actions += ["LOCK user account", "Revoke active sessions",
                    "Notify SOC team", "Preserve mail logs"]
    if incident['type'] == 'phishing':
        actions += ["Quarantine email", "Block sender domain",
                    "Scan attachments in sandbox"]

    print("\nActions Taken:")
    for a in actions:
        print(f"  [x] {a}")

    report = {"incident": incident, "actions": actions,
              "timestamp": str(datetime.datetime.now())}
    with open("ir_report.json","w") as f:
        json.dump(report, f, indent=2)
    print("\nIR report saved: ir_report.json")

ir_response({"type":"phishing","severity":"HIGH","user":"riya@sqrock.com"})
```

■ Tools: json (stdlib) | datetime (stdlib)

■ Deliverable: IR script + JSON report sample + 1-page IR playbook document.

DAY 15

Final Project: SE Attack Chain Simulator

■ Expert

■ OBJECTIVE

Build a complete social engineering attack chain demo tool (ethical lab use).

■ THEORY

- Attack chain: OSINT → Profile Build → Phish Craft → Delivery → Exploit → Persist.
- Red team exercises expose gaps before real attackers do.
- Every component built this week comes together in a full simulation.

■ TASK

Combine OSINT scanner + phish scorer + email template + IR trigger into one CLI tool.

</> PYTHON

```
# se_chain.py — Full SE Simulation Chain
import sys

MODULES = {
    "osint"   : "Run passive OSINT on a domain",
    "profile" : "Build target profile from public data",
    "phish"   : "Score a URL for phishing indicators",
    "template": "Generate spear-phishing training email",
    "ir"      : "Trigger incident response workflow",
}

def menu():
    print("\n■■■ SE CHAIN SIMULATOR ■■■■■■■■■■■■■■■■■■■■")
    print("■  Sqrock Cybersecurity Internship       ■")
    print("■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■")
    for k, v in MODULES.items():
        print(f"  [{k}]  {v}")
    choice = input("\nSelect module: ").strip().lower()
    if choice in MODULES:
        print(f"\n[+] Launching {choice} module...")
        # Each module is imported from previous days' scripts
    else:
        print("Invalid choice.")
    menu()

if __name__ == "__main__":
    menu()
```

■ Tools: All previous tools combined

■ Deliverable: Full integrated tool + live demo recording + final security report (PDF).

■ ETHICAL & LEGAL GUIDELINES

Rule	Explanation
Authorized Systems ONLY	All tasks must be performed on lab VMs, your own domains, or explicitly authorized systems. No exceptio
No Real Targets	Never run phishing, harvesting, or scanning against real users, companies, or infrastructure.
Data Privacy	Any data collected during OSINT exercises must be anonymized or deleted after analysis.
Report Vulnerabilities	If you discover real vulnerabilities accidentally, report to your supervisor immediately.
Documentation	Maintain a lab journal. All experiments must be logged with timestamps and scope notes.
Legal Compliance	Activities without written authorization may violate IT Act 2000 (India), CFAA (US), and GDPR. You are p

■ GRADING RUBRIC

Component	Weight	Criteria
Working Python Script	40%	Code runs, produces correct output, is well-commented
Written Report / Analysis	30%	Depth of explanation, attacker vs defender perspective
Deliverable Submitted	20%	Screenshot / output file / JSON report as specified
Code Quality	10%	PEP-8 style, error handling, no hardcoded secrets

By participating in this program, interns acknowledge they have read and agree to the ethical guidelines above. Sqrock IT Solution reserves the right to revoke internship credentials for any unauthorized activity.